Advanced Database Systems Portfolio

David-Paul Folorunsho-Roberts | Student ID 2026/A/MIT/0268 | MIT 8103 Advanced Database Systems | 2026/2027 First Semester
GitHub repository: github.com/davidcreated/advanced-database-portfolio (private, with access granted to the assessor)

Selected Organisation: OjaHub Marketplace

OjaHub is a simulated Nigerian online marketplace ("oja" means market in Yoruba). Independent sellers open a shop and list products, and customers in all 36 states and the FCT order, pay and receive deliveries through courier partners. I used it as the single case study across all five activities.

Users of the database: customers (browse, order, pay, review), sellers (manage products and stock), operations staff (payments, shipments, disputes) and management (sales reports by region, category and seller).

Major data: customers and addresses, sellers, product categories, products and stock, orders and order items, payments (including failed attempts), shipments and reviews.

Key operations: register users, manage products and stock, place an order and reduce stock safely when many people buy at once, process payments and retries, track delivery, and produce sales reports.

Portfolio 1: Database Design and Modelling

I designed an 11 table relational schema in PostgreSQL 18 and walked a flat order spreadsheet through 1NF, 2NF and 3NF, then checked every table against BCNF. Business rules live in the database as primary, foreign and unique keys and CHECK constraints, plus two partial unique indexes: one default address per customer, and one successful payment per order while still allowing failed retries. All 14 rule breaking test statements were rejected by the exact constraint they targeted, and a valid retry was accepted. Evidence: ER diagram, schema, normalisation write up and constraint test output.

Portfolio 2: Query Processing and Optimisation

I generated about 2.9 million deterministic rows (500,010 orders, 1.07 million order items) and benchmarked eight business queries with `EXPLAIN (ANALYZE, BUFFERS)`, reporting the median of six warm runs.

Query	Before	After	What fixed it
Customer order history	11.44 ms	0.08 ms	Index on the unindexed foreign key
One product's sales	38.87 ms	0.09 ms	Index on the unindexed foreign key
Product name search	40.89 ms	1.74 ms	Trigram GIN index
Stuck pending orders	56.95 ms	0.14 ms	Partial index (56 kB against 19 MB for a full one)
Monthly sales report	2,109 ms	0.06 ms	Materialised view (refresh about 1.7 s)

Indexes did nothing for the reporting queries, and the plans showed why. They also made writes about 23 percent slower, which I measured too.

Portfolio 3: Transactions and Concurrency

Using dblink to script several real sessions in a fixed order, I showed commit, rollback, a transaction poisoned by an error, and a savepoint. Two buyers racing for the last Standing Fan under READ COMMITTED both succeeded, so one fan was sold twice. `SELECT ... FOR UPDATE`, an atomic conditional UPDATE and REPEATABLE READ each fixed it with exactly one order. A report saw phantom and non repeatable reads under READ COMMITTED but a stable snapshot under REPEATABLE READ. Write skew broke a business rule under REPEATABLE READ and was rejected under SERIALIZABLE (SQLSTATE 40001). A deadlock was detected and resolved (40P01), then prevented by locking in a fixed order.

Portfolio 4: NoSQL and Advanced Data Models

I modelled OjaHub as MongoDB documents: products with category specific attributes and a subset of recent reviews, and orders embedding their items, payments and shipment as a snapshot of the day. All 760,088 documents migrated from PostgreSQL and passed `$jsonSchema` validation, with counts and spot checked totals matching. Indexing cut a customer's order history from scanning 500,010 documents (80.32 ms) to reading 10 (0.39 ms). With ten concurrent buyers, a naive read and write sold one fan ten times, while one atomic `findOneAndUpdate` sold it once. Compared side by side, MongoDB served the order page faster (0.54 ms against 1.16 ms) and PostgreSQL won substring search (1.52 ms against 3.96 ms). My conclusion is polyglot persistence: MongoDB for the catalogue and order pages, PostgreSQL for money, stock and reporting.

Portfolio 5: Distributed and Cloud Database Exercise

I ran real clusters locally and wrote a cloud deployment design from the results.

Setup	Key result
PostgreSQL streaming replica	Lag peaked near 8 MB and 0.19 s during a million row write. Synchronous commits rose from 0.089 ms to 0.143 ms, and with the standby down a commit waited until cancelled. Promotion took 133 ms and showed split brain
MongoDB three member replica set	Killing the primary stopped writes for 10.4 s, with 0 of 173 acknowledged writes lost. Reads from a secondary were stale 126 times in 300, and 0 in a causally consistent session
MongoDB sharded cluster	Zone sharding put 209,572 northern orders on one shard and 290,438 southern orders on the other. Region queries hit one shard, while customer queries had to ask both

The design places payments and stock on PostgreSQL with a synchronous standby in a second availability zone, and the catalogue on a majority write MongoDB replica set. It also covers data residency, backups, and a CAP and PACELC reading of each choice.

Use of AI

To be completed by the student before submission.

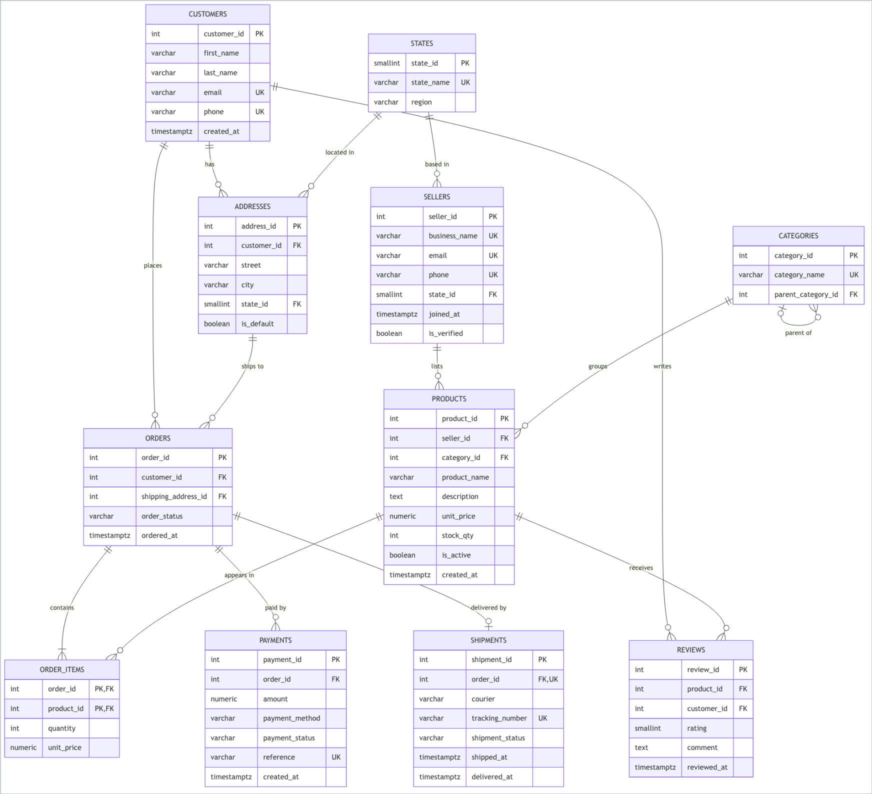
Individual Technical Reflection

Design choices. I chose an online marketplace because it carries every topic in the course: money, stock that runs out, flexible product data and regional customers. The choice I am happiest with was storing the price paid on each order item separately from the product's list price. It looked like duplication in Portfolio 1, but the same idea came back as the order snapshot in the MongoDB model.

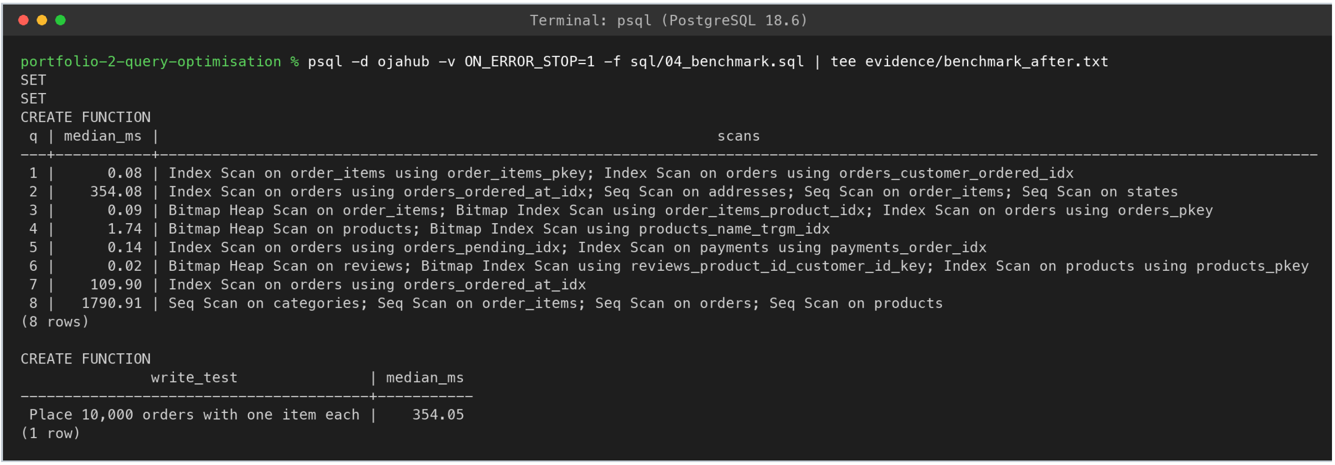
Challenges. The hardest part was making the results honest. Heavy queries varied by 30 to 60 percent between runs, so I had to compare medians and only trust large differences. Twice my laptop went to sleep on battery in the middle of a run and made the database look broken, until I checked the power log. Some of my predictions were also wrong: the planner used an index I expected it to ignore, and a synchronous commit hung even with a statement timeout.

Lessons learned. Read the execution plan before adding an index. A transaction is only as safe as the logic inside it, as the lost update showed. Denormalising and replicating data are trade offs that have to be chosen on purpose, not free speed. Seeing a 10 second election, a split brain and 126 stale reads made consistency and availability feel concrete rather than theoretical.

Appendix: Selected Evidence

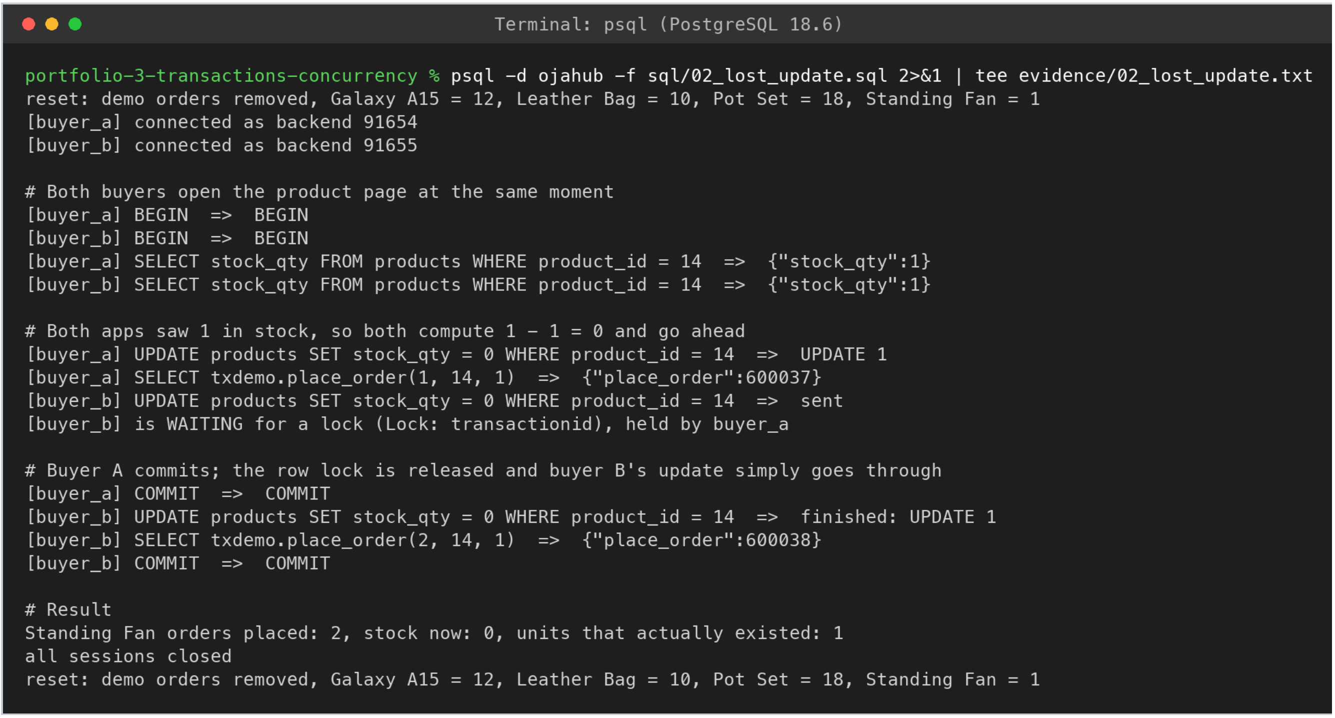


Portfolio 1: ER diagram of the OjaHub schema in crow's foot notation.



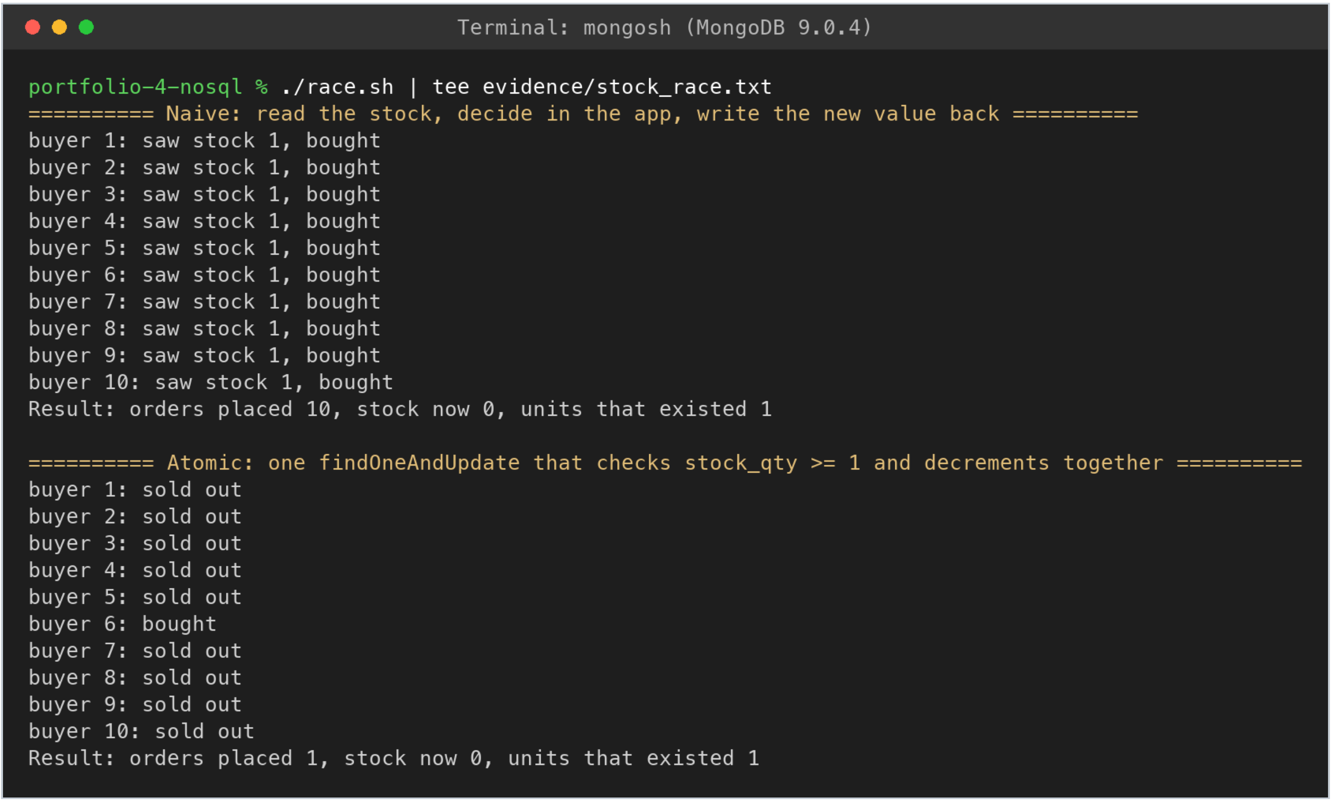
```
portfolio-2-query-optimisation % psql -d ojahub -v ON_ERROR_STOP=1 -f sql/04_benchmark.sql | tee evidence/benchmark_after.txt
SET
SET
CREATE FUNCTION
 q | median_ms |                                                                scans
---+-----------+-------------------------------------------------------------------------------------------------------------------------------------
 1 |      0.08 | Index Scan on order_items using order_items_pkey; Index Scan on orders using orders_customer_ordered_idx
 2 |    354.08 | Index Scan on orders using orders_ordered_at_idx; Seq Scan on addresses; Seq Scan on order_items; Seq Scan on states
 3 |      0.09 | Bitmap Heap Scan on order_items; Bitmap Index Scan using order_items_product_idx; Index Scan on orders using orders_pkey
 4 |      1.74 | Bitmap Heap Scan on products; Bitmap Index Scan using products_name_trgm_idx
 5 |      0.14 | Index Scan on orders using orders_pending_idx; Index Scan on payments using payments_order_idx
 6 |      0.02 | Bitmap Heap Scan on reviews; Bitmap Index Scan using reviews_product_id_customer_id_key; Index Scan on products using products_pkey
 7 |    109.90 | Index Scan on orders using orders_ordered_at_idx
 8 |   1790.91 | Seq Scan on categories; Seq Scan on order_items; Seq Scan on orders; Seq Scan on products
(8 rows)

CREATE FUNCTION
               write_test               | median_ms
----------------------------------------+-----------
 Place 10,000 orders with one item each |    354.05
(1 row)
```

Portfolio 2: benchmark after indexing, with medians and the scan type chosen for each query.



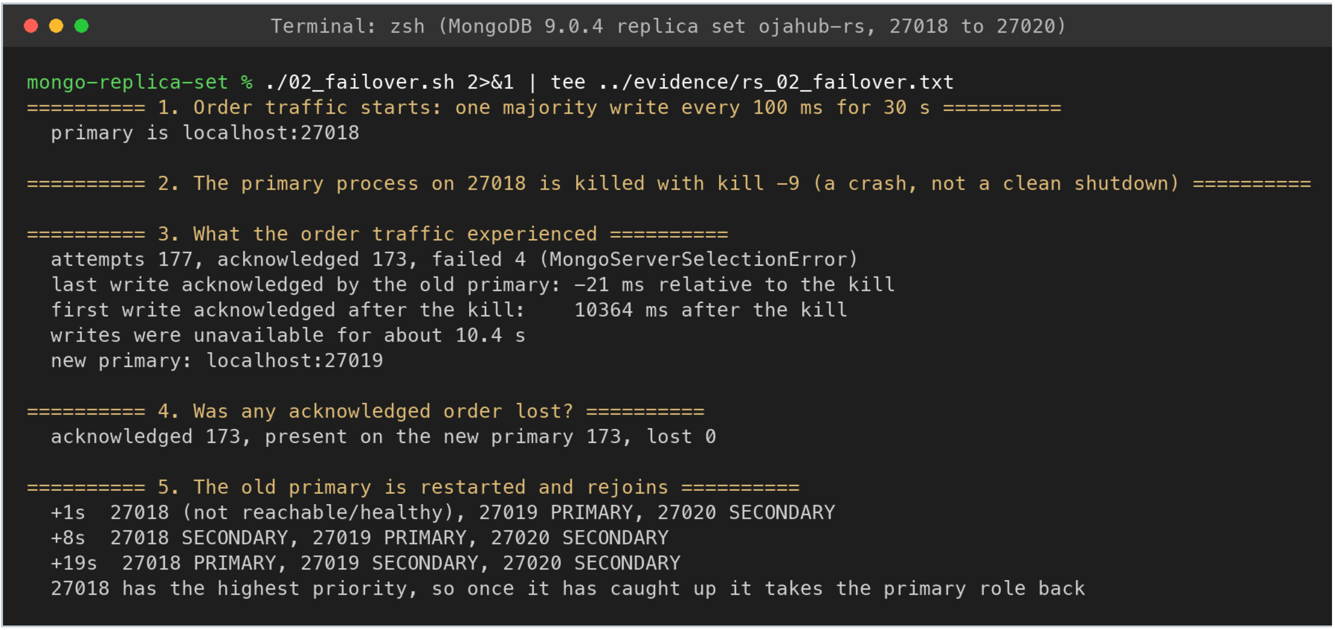
```
portfolio-3-transactions-concurrency % psql -d ojahub -f sql/02_lost_update.sql 2>&1 | tee evidence/02_lost_update.txt
reset: demo orders removed, Galaxy A15 = 12, Leather Bag = 10, Pot Set = 18, Standing Fan = 1
[buyer_a] connected as backend 91654
[buyer_b] connected as backend 91655

# Both buyers open the product page at the same moment
[buyer_a] BEGIN  =>  BEGIN
[buyer_b] BEGIN  =>  BEGIN
[buyer_a] SELECT stock_qty FROM products WHERE product_id = 14  =>  {"stock_qty":1}
[buyer_b] SELECT stock_qty FROM products WHERE product_id = 14  =>  {"stock_qty":1}

# Both apps saw 1 in stock, so both compute 1 - 1 = 0 and go ahead
[buyer_a] UPDATE products SET stock_qty = 0 WHERE product_id = 14  =>  UPDATE 1
[buyer_a] SELECT txdemo.place_order(1, 14, 1)  =>  {"place_order":600037}
[buyer_b] UPDATE products SET stock_qty = 0 WHERE product_id = 14  =>  sent
[buyer_b] is WAITING for a lock (Lock: transactionid), held by buyer_a

# Buyer A commits; the row lock is released and buyer B's update simply goes through
[buyer_a] COMMIT  =>  COMMIT
[buyer_b] UPDATE products SET stock_qty = 0 WHERE product_id = 14  =>  finished: UPDATE 1
[buyer_b] SELECT txdemo.place_order(2, 14, 1)  =>  {"place_order":600038}
[buyer_b] COMMIT  =>  COMMIT

# Result
Standing Fan orders placed: 2, stock now: 0, units that actually existed: 1
all sessions closed
reset: demo orders removed, Galaxy A15 = 12, Leather Bag = 10, Pot Set = 18, Standing Fan = 1
```

Portfolio 3: the lost update, two orders placed for one Standing Fan under READ COMMITTED.



```
portfolio-4-nosql % ./race.sh | tee evidence/stock_race.txt
========== Naive: read the stock, decide in the app, write the new value back ==========
buyer 1: saw stock 1, bought
buyer 2: saw stock 1, bought
buyer 3: saw stock 1, bought
buyer 4: saw stock 1, bought
buyer 5: saw stock 1, bought
buyer 6: saw stock 1, bought
buyer 7: saw stock 1, bought
buyer 8: saw stock 1, bought
buyer 9: saw stock 1, bought
buyer 10: saw stock 1, bought
Result: orders placed 10, stock now 0, units that existed 1

========== Atomic: one findOneAndUpdate that checks stock_qty >= 1 and decrements together ==========
buyer 1: sold out
buyer 2: sold out
buyer 3: sold out
buyer 4: sold out
buyer 5: sold out
buyer 6: bought
buyer 7: sold out
buyer 8: sold out
buyer 9: sold out
buyer 10: sold out
Result: orders placed 1, stock now 0, units that existed 1
```

Portfolio 4: ten concurrent buyers in MongoDB, naive read and write against one atomic update.


Terminal: zsh (MongoDB 9.0.4 replica set ojahub-rs, 27018 to 27020)

```
mongo-replica-set % ./02_failover.sh 2>&1 | tee ../evidence/rs_02_failover.txt
========== 1. Order traffic starts: one majority write every 100 ms for 30 s ==========
  primary is localhost:27018

========== 2. The primary process on 27018 is killed with kill -9 (a crash, not a clean shutdown) ==========

========== 3. What the order traffic experienced ==========
  attempts 177, acknowledged 173, failed 4 (MongoServerSelectionError)
  last write acknowledged by the old primary: -21 ms relative to the kill
  first write acknowledged after the kill:    10364 ms after the kill
  writes were unavailable for about 10.4 s
  new primary: localhost:27019

========== 4. Was any acknowledged order lost? ==========
  acknowledged 173, present on the new primary 173, lost 0

========== 5. The old primary is restarted and rejoins ==========
  +1s  27018 (not reachable/healthy), 27019 PRIMARY, 27020 SECONDARY
  +8s  27018 SECONDARY, 27019 PRIMARY, 27020 SECONDARY
  +19s  27018 PRIMARY, 27019 SECONDARY, 27020 SECONDARY
  27018 has the highest priority, so once it has caught up it takes the primary role back
```

Portfolio 5: replica set failover under write traffic, 10.4 s without writes and no acknowledged write lost.